FIE Security Scanner

Setup, Deployment, and User Guide

VMware Fusion • Ubuntu • Chromium • Python venv • Node/npm • SSH bridge

What this guide covers
Host machine setup on macOS.
Creating and configuring an Ubuntu VM in VMware Fusion.
Installing Chromium and project dependencies inside the VM.
Installing Node.js and npm on the local machine for the website.
Creating an SSH connection so the website can trigger VM downloads.
Cloning the repository, launching the software, and using the scanner.

1. System overview

The FIE Security Scanner is a split setup: the web interface runs on the local machine, while extension downloading and analysis run inside an Ubuntu virtual machine. The two sides communicate over SSH. When a user searches for an extension or pastes a Chrome Web Store URL into the website, the local Node server uses SSH to call a downloader or manager script inside the VM. The VM downloads the extension, extracts it, runs the parsing and analysis pipeline, and sends the results back to the local website for display.

2. Architecture at a glance

- Local machine (macOS): runs the website, Node.js, npm packages, and the browser used to open the UI.
- VMware Fusion: hosts the Ubuntu virtual machine.
- Ubuntu VM: runs Chromium, Python, a virtual environment, the parser, model code, and downloader scripts.
- SSH connection: lets the local website call commands inside Ubuntu.
- GitHub repository: stores the website code, parser, machine-learning logic, and helper scripts.

3. Before you begin

- A Mac with enough free disk space for Ubuntu, dependencies, and extracted extensions. At least 30 GB free is recommended.
- At least 8 GB RAM on the host machine; 16 GB is more comfortable if VMware and the website will run together.
- Internet access.
- A GitHub account if the repository is private.
- Permission to install software such as VMware Fusion, Xcode command line tools, Homebrew, Node.js, and Ubuntu packages.

4. Local machine setup on macOS

The local machine is where the website is started. This machine needs Git, Node.js, npm, and an SSH client.

4.1 Install Apple command line tools

```
xcode-select --install
```

This installs the command line tools needed for Git and other build steps.

4.2 Install Homebrew (recommended)

```
/bin/bash -c "$(curl -fsSL
https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

Homebrew is the easiest way to install Git and Node.js on macOS if they are not already available.

4.3 Install Git

```
brew install git
```

4.4 Install Node.js and npm

```
brew install node

node -v
npm -v
```

The website portion of the project depends on Node.js and npm. If the repository already includes package.json, npm install will pull the required node modules.

5. Install VMware Fusion

Download VMware Fusion from VMware/Broadcom and install it in Applications. Approve any macOS prompts. When you create the VM, give it enough resources to run Ubuntu, Python dependencies, and parser jobs comfortably.

- Memory: 4 GB minimum, 6 to 8 GB recommended.
- CPUs: 2 cores minimum.
- Disk: 30 GB minimum.
- Networking: NAT is usually easiest. Bridged networking is optional if your team specifically wants the VM to appear directly on the local network.

6. Install Ubuntu in VMware Fusion

- Download the Ubuntu Desktop ISO.
- Create a new VM in VMware Fusion and choose the Ubuntu ISO as the installation image.
- Create an Ubuntu username and password. Use a username you will remember because it will also appear in SSH commands and file paths. Example: fiecomps.
- Finish installation and boot into Ubuntu.
- Open Terminal in Ubuntu and update the system.

```
sudo apt update && sudo apt upgrade -y
```

- Take a VM snapshot after the clean Ubuntu install. This makes recovery easier if later dependency changes break the environment.

7. Install Chromium inside Ubuntu

Chromium is useful for extension-related workflows and for reproducing browser behavior inside the VM if needed.

```
sudo apt install -y chromium-browser || sudo apt install -y chromium
```

If neither package name works, run apt search chromium and install the package available for your Ubuntu version.

8. Install Ubuntu packages needed for development

```
sudo apt install -y python3 python3-pip python3-venv git curl unzip
openssh-server build-essential
```

These packages cover Python, virtual environments, Git, SSH server support, and common build tools.

9. Clone the repository

Clone the project repository in the environment where the code will actually run. In this setup, that usually means cloning inside the Ubuntu VM first, and optionally cloning on the local Mac as well if the website is run locally from a separate checkout.

```
git clone <YOUR_GITHUB_REPO_URL>
cd <YOUR_REPO_FOLDER>
```

If you need a specific branch:

```
git checkout <BRANCH_NAME>
```

10. Create a Python virtual environment inside the VM

The parser and machine-learning side of the project should run inside a Python virtual environment in Ubuntu. This keeps project packages separate from the system Python installation.

```
cd <YOUR_REPO_FOLDER>
python3 -m venv .venv
source .venv/bin/activate
```

After activation, your terminal prompt should show the virtual environment name, often (.venv).

10.1 Install Python dependencies from requirements.txt

```
pip install --upgrade pip
pip install -r requirements.txt
```

If the project uses multiple requirements files, install each one in the order your repository expects.

11. Set up the local website with Node.js and npm

The website is usually launched from the local Mac. Open Terminal on macOS, move into the website or project folder, and install the node modules defined by package.json.

```
cd <LOCAL_CHECKOUT_OF_REPO_OR_WEB_FOLDER>
npm install
```

This downloads the node_modules folder and any JavaScript dependencies needed by the UI and server.

11.1 Start the local website

Depending on your repository, the start command may be npm start or node server.js.

```
npm start
```

or

```
node server.js
```

12. Set up SSH between the local machine and the Ubuntu VM

This is the critical bridge that allows the website to trigger downloads on the VM. The local machine must be able to SSH into Ubuntu without interactive prompts, ideally using an SSH key.

12.1 Enable the SSH server in Ubuntu

```
sudo systemctl enable ssh
sudo systemctl start ssh
sudo systemctl status ssh
```

The status command should show that the SSH service is active and running.

12.2 Find the VM IP address

```
hostname -I
```

Write down the VM IP address. If you are using NAT, this is often a private VMware network address. This IP must be placed into the local server configuration so the website knows where to connect.

12.3 Test manual SSH from the Mac

```
ssh <VM_USERNAME>@<VM_IP_ADDRESS>
```

Example: ssh fiecomps@192.168.217.128

If this works, your network path is correct. If it fails, check that the VM is running, the SSH service is active, and the IP address is correct.

12.4 Create an SSH key on the Mac

```
ssh-keygen -t ed25519 -C "fie-scanner-local"
```

Press Enter to accept the default location unless your team uses a different SSH key naming convention.

12.5 Copy the public key into the VM

```
ssh-copy-id <VM_USERNAME>@<VM_IP_ADDRESS>
```

If ssh-copy-id is unavailable on your Mac, manually append the contents of ~/.ssh/id_ed25519.pub into ~/.ssh/authorized_keys inside the VM user account.

12.6 Confirm passwordless SSH

```
ssh <VM_USERNAME>@<VM_IP_ADDRESS> "echo SSH_OK"
```

If the terminal returns SSH_OK without asking for a password, the website will be able to execute remote commands more reliably.

13. Connect the website configuration to the VM

Your Node server or local backend must know the SSH details required to reach Ubuntu. These values often appear directly in server.js and need to be changed/updated with each new VM.

Setting	Example value	Where to check	Why it matters
VM username	fiecomps	Ubuntu VM user account and SSH commands	The local website uses this to log into the VM.

VM IP address	192.168.xxx.xxx	server.js or config file	The Node server must know where the VM lives on the network.
Remote downloader path	/home/fiecomps/vm_downloader.py	server.js helper function	The website triggers this script over SSH.
Repo branch	main or your team branch	git branch / git checkout	Ensures the website and parser are using the intended code version.
Frontend port	3000	Node app start command	The user opens this in the browser.

In code, this usually appears as a helper that runs a remote command over SSH. Make sure the username, IP address, and remote script path all match the actual Ubuntu environment you created.

```
const VM_USER = "<VM_USERNAME>";
const VM_IP = "<VM_IP_ADDRESS>";
const VM_DOWNLOADER = "/home/<VM_USERNAME>/vm_downloader.py";
```

14. Launch workflow

14.1 Inside the VM

```
cd <YOUR_REPO_FOLDER>
source .venv/bin/activate
```

Keep the VM running. If your Python backend must also be started manually, start it here according to your repository instructions.

14.2 On the local Mac

```
cd <LOCAL_CHECKOUT_OF_REPO_OR_WEB_FOLDER>
npm install
npm start
```

or, if your project uses a direct Node entry point:

```
node server.js
```

14.3 Open the UI

Open the local website in a browser, typically at http://localhost:3000 unless your project uses a different port.

15. How a user actually uses the software

- Start the Ubuntu VM in VMware Fusion.
- Activate the Python virtual environment in the VM if the backend or parser requires it.
- Start the local website on the Mac with npm start or node server.js.

- Open the scanner website in the browser.
- Search for a Chrome extension by name or paste a Chrome Web Store URL.
- Submit the request.
- The website uses SSH to trigger the downloader script on the VM.
- The VM downloads the extension, extracts the contents, runs parsing and analysis, and returns structured results.
- Review the output shown in the UI, including any risk or analysis information produced by the model.

16. Suggested startup checklist for demos

- VM is powered on.
- Ubuntu has internet access.
- ssh <VM_USERNAME>@<VM_IP_ADDRESS> "echo SSH_OK" works from the local Mac.
- The Python virtual environment exists and requirements.txt has been installed.
- The vm_downloader file is copied to the home directory along with download_mal_ext.
- The local website folder has completed npm install.
- The correct repo branch is checked out.
- The local UI opens successfully at localhost.

17. Troubleshooting

17.1 Website cannot reach the VM

- Confirm the VM is running.
- Run hostname -I inside Ubuntu and verify the configured IP address still matches.
- Restart SSH inside Ubuntu with sudo systemctl restart ssh.
- Run a direct SSH test from the Mac.
- Make sure VMware networking mode has not changed since the IP was configured.

17.2 ModuleNotFoundError inside the parser

- Activate the virtual environment first: source .venv/bin/activate.
- Reinstall dependencies with pip install -r requirements.txt.
- Install the missing package inside the venv, not globally.

17.3 npm or node_modules issues

- Delete node_modules and package-lock.json only if necessary, then rerun npm install.
- Confirm the Node.js version is compatible with the project.

17.4 Search works but URL analysis fails, or vice versa

- Check whether the problem is in the local website logic or in the remote downloader path.
- Inspect server logs on the Mac and parser/downloader logs in the VM.
- Verify that manager.py or the remote script outputs valid JSON if the UI expects JSON.

18. Items to customize for your final team handoff

- Replace <YOUR_GITHUB_REPO_URL> with the real repository URL.
- Replace placeholder branch names with the branch your team wants evaluators to use.
- Replace the remote script path with the actual script used in your project.

- Add the exact start commands used by your final website and backend.
- If your team uses a specific VM username or static IP, write those values explicitly.

19. One-page quick start

Minimal run sequence
1. Start VMware Fusion and boot the Ubuntu VM.
2. In Ubuntu: cd into the repo, source .venv/bin/activate.
3. On the Mac: cd into the website folder and run npm install if needed.
4. Start the website with npm start or node server.js.
5. Confirm SSH works: ssh <VM_USERNAME>@<VM_IP_ADDRESS> "echo SSH_OK".
6. Open http://localhost:3000 and search for an extension or paste a store URL.
7. Review the results returned by the scanner.